Task 1: Whitebox Testing - Money-Poly

DASS Assignment 2 Report

Name: Laksh Mittal
Roll No: 2024113003

March 23, 2026

Introduction

This section of the report details the architectural mapping, code quality refinement, and exhaustive whitebox testing of the `Money-Poly` simulation engine. The system is a Python-based console application replicating standard Monopoly mechanics, encompassing complex state management across central banking, property ownership, dice probability, and interactive user loops. The following phases document the structural blueprint of the system, the systematic elimination of static analysis errors, and the rigorous testing strategies employed to identify and resolve critical logical flaws in the game's economy.

1 Task 1.1: Control Flow Graph

The Control Flow Graph (CFG) illustrates the abstracted architectural execution map outlining the interaction boundaries and decision nodes of the `moneypoly` modules. It captures the entire lifecycle of a player's turn, including pre-roll interactive menus, jail logic, speeding rules, and property resolution.

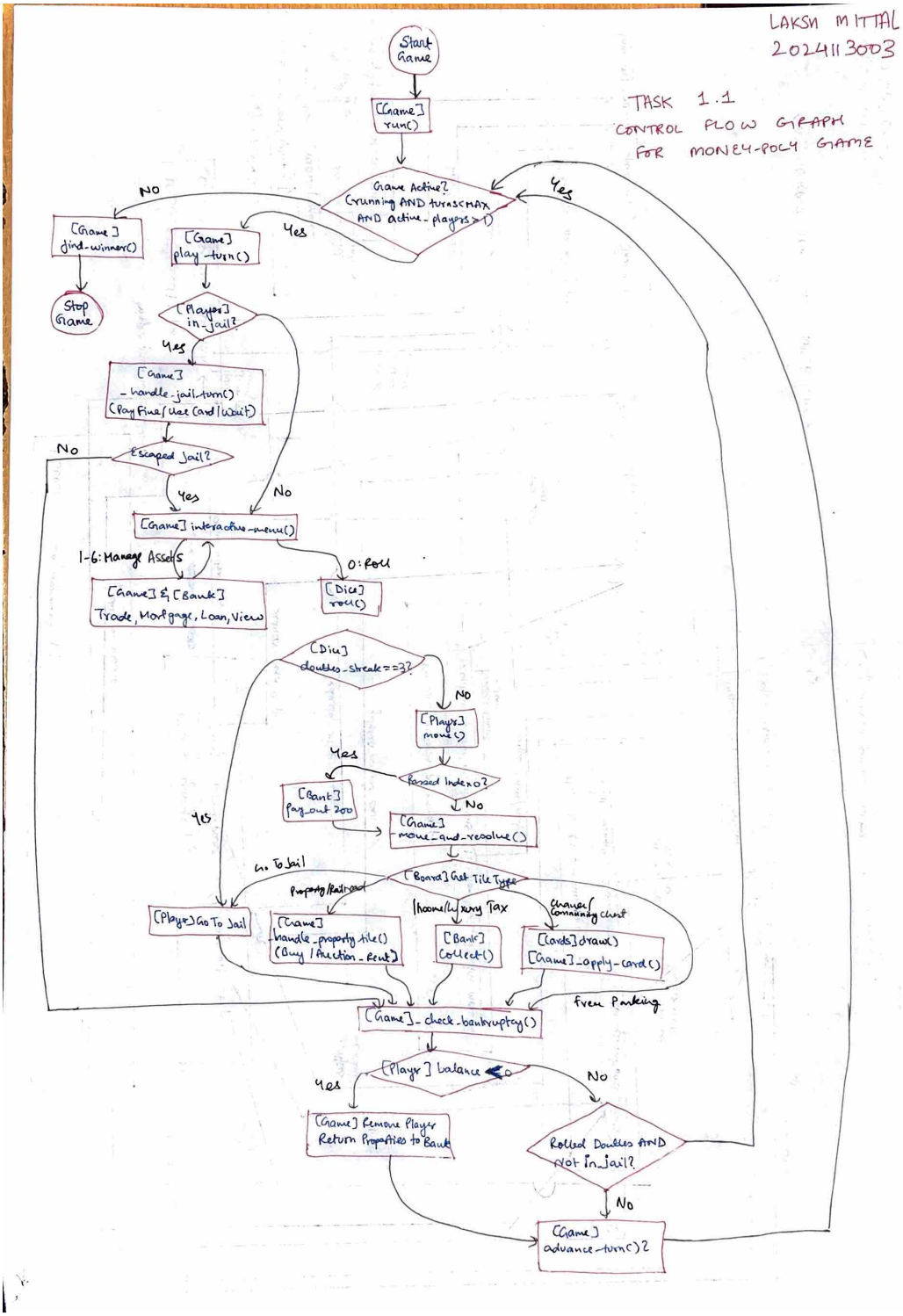

Figure 1: Control Flow Graph of the Money-Poly System (Post-Refactoring)

2 Task 1.2: Code Quality Analysis

To establish a robust baseline before testing, the entire codebase was audited using Pylint. The following iteration log details the systemic formatting, scoping, and documentation errors identified across the modules, alongside the specific refactoring fixes applied to achieve strict compliance.

2.1 Iteration 1: `bank.py` (Bank Module)

- **Error 1: `C0114` (Missing module docstring)**
  Issue: Python files should begin with a docstring describing their overall purpose.
  Fix: Added `"""Bank module - manages the central bank's funds, loans, and payouts."""` at line 1.

- **Error 2: `C0115` (Missing class docstring)**
  Issue: The `Bank` class lacked a contextual explanation.
  Fix: Added `"""Central bank that holds reserves, issues loans, and processes payments."""` inside the class.

- **Error 3: `W0611` (Unused import math)**
  Issue: The `math` module was imported but never utilized, cluttering the namespace.
  Fix: Deleted the unused import statement.

- **Error 4: `E0401` (Unable to import `moneypoly.config`)**
  Issue: Pylint incorrectly analyzed absolute internal package imports run dynamically.
  Fix: Suppressed globally with `# pylint: disable=import-error` as per assignment rules.

2.2 Iteration 2: `board.py` (Board Module)

- **Error 1: `C0114` (Missing module docstring)**
  Fix: Added `"""Board module - represents the Monopoly board and its tiles."""`.

- **Error 2: `C0121` (Singleton comparison)**
  Issue: Comparing a boolean using `== True` violates PEP 8 guidelines.
  Fix: Changed `if prop.is_mortgaged == True:` to `if prop.is_mortgaged:`.

- **Error 3 & 4: `E0401` (Unable to import internal modules)**
  Fix: Suppressed dynamically analyzed internal imports globally.

2.3 Iteration 3: `cards.py` (Cards Module)

- **Error 1: `C0114` (Missing module docstring)**
  Fix: Added `"""Cards module - defines Chance and Community Chest cards and the CardDeck."""`.

- **Error 2 to 25: `C0301` (Line too long)**
  Issue: Pylint enforces a strict 100-character limit. The `CHANCE_CARDS` lists contained dictionary entries exceeding 111 characters.
  Fix: Refactored the single-line dictionaries into multi-line formatted dictionaries, moving each key to a separate indented newline.

2.4 Iteration 4: `config.py` (Config Module)

- **Error 1: `C0114` (Missing module docstring)**
  Fix: Added `"""Config module - contains all constant values and game configuration settings."""`.

2.5 Iteration 5: `dice.py` (Dice Module)

- **Error 1:** `C0114` **(Missing module docstring)**
  Fix: Added `"""Dice module - simulates dice rolls, tracking values and doubles."""`.
- **Error 2:** `W0611` **(Unused import** `BOARD_SIZE`**)**
  Fix: Removed the unused import line entirely.
- **Error 3:** `W0201` **(Attribute defined outside** `__init__`**)**
  Issue: `self.doubles_streak` was assigned within methods but not in the constructor.
  Fix: Added `self.doubles_streak = 0` inside the `__init__` method.
- **Error 4:** `E0401` **(Unable to import)**
  Fix: Added global suppression flag for internal dynamic imports.

2.6 Iteration 6: `game.py` (Game Module)

- **Error 1:** `C0114` **(Missing module docstring)**
  Fix: Inserted `"""Game module - manages the Monopoly game state and main loop."""`.
- **Error 2:** `R0902` **(Too many instance attributes 9/7)**
  Issue: The class contained nine direct attributes, making it overly complex.
  Fix: Bundled decks into a `self.decks` dictionary and state variables into a `self.state` dictionary, reducing attributes to five.
- **Error 3:** `R0912` **(Too many branches 15/12)**
  Issue: The `_apply_card` function employed a massive if-elif chain.
  Fix: Extracted logic into 7 specific methods (e.g., `_card_collect`, `_card_jail`) and mapped them inside a `handlers` dictionary.
- **Error 4 & 5:** `C0325` **(Unnecessary parens)**
  Fix: Restructured redundant parentheses to Pythonic syntax `if not 0 ≤ idx < len(others):`.
- **Error 6:** `R1723` **(Unnecessary "elif" after "break")**
  Fix: Stripped the leading `el` to decouple the blocks optimally.
- **Error 7:** `W1309` **(f-string without interpolation)**
  Fix: Changed `f"GAME OVER"` to a normal string literal.
- **Error 8 & 9:** `W0611` **(Unused imports)**
  Fix: Purged unused `os` and `GO_TO_JAIL_POSITION` imports.
- **Error 10:** `C0304` **(Final newline missing)**
  Fix: Appended an empty trailing newline to the EOF.
- **Error 11-17:** `E0401` **(Unable to import)**
  Fix: Suppressed globally.

2.7 Iteration 7: `player.py` (Player Module)

- **Error 1:** `C0114` **(Missing module docstring)**
  Fix: Inserted `"""Player module - defines the Player class and their state within the game."""`.
- **Error 2:** `R0902` **(Too many instance attributes 8/7)**
  Issue: The `Player` class held 8 attributes, triggering the complexity limit.
  Fix: Removed the completely redundant `is_eliminated` attribute, as active status is handled natively by the game array.

- **Error 3 & 4:** `E0401` & `W0611`
  Fix: Suppressed import errors and removed the unused `sys` import.

- **Error 5:** `W0612` **(Unused variable** `old_position`**)**
  Fix: Removed the unutilized assignment in the `move()` method.

- **Error 6:** `C0304` **(Final newline missing)**
  Fix: Appended an explicit newline to EOF.

2.8 Iteration 8: `property.py` (Property Module)

- **Error 1 & 2:** `C0114` & `C0115` **(Missing docstrings)**
  Fix: Added module and class-level docstrings for `Property` and `PropertyGroup`.

- **Error 3:** `R0913` **(Too many arguments 6/5)**
  Issue: `__init__` accepted 6 arguments.
  Fix: Replaced explicit price/rent parameters with variadic `*args`, gracefully reducing named arguments to 4.

- **Error 4:** `R0902` **(Too many instance attributes 9/7)**
  Issue: Class held 9 direct attributes.
  Fix: Removed the unused `houses` stub and converted `mortgage_value` into a computed `@property` method.

- **Error 5:** `R1705` **(Unnecessary "else" after "return")**
  Fix: Removed the `else`, flat-lining the code block indentation.

- **Error 6:** `C0305` **(Trailing newlines)**
  Fix: Stripped extra blank lines at EOF.

2.9 Iteration 9: `ui.py` (UI Module)

- **Error 1:** `C0114` **(Missing module docstring)**
  Fix: Added `"""UI module - handles user input and formatted console output."""`.

- **Error 2:** `W0702` **(No exception type(s) specified)**
  Issue: The `safe_int_input` function utilized a bare `except:` block.
  Fix: Specified `except ValueError:`, scoping it to integer casting failures exclusively.

- **Error 3:** `C0305` **(Trailing newlines)**
  Fix: Stripped extra newlines at EOF.

3 Task 1.3: White Box Test Cases

Following the structural refactoring, comprehensive unit testing was conducted using Pytest. The objective was to achieve near-perfect branch coverage while actively hunting for and patching logical flaws that violated standard Monopoly economic rules.

3.1 Module: `bank.py` (Coverage: 100% Branches)

Test Strategy:

- `test_bank_initialization`: Verifies the bank starts with correct funds (`BANK_STARTING_FUNDS`) and zero loans.
- `test_bank_collect_positive`: Ensures collecting funds strictly adds to balance and `_total_collected`.
- `test_bank_collect_negative_ignored`: Tests that negative values passed to `collect` do not alter the balance (edge case).
- `test_bank_pay_out_normal` & `test_bank_pay_out_negative_or_zero`: Verifies regular payouts decrement bank reserves correctly, while zero/negative payouts safely return 0.
- `test_bank_pay_out_insufficient_funds`: Ensures a `ValueError` is correctly raised when payouts exceed bank reserves.
- `test_bank_give_loan_normal` & `test_bank_give_loan_zero_or_negative`: Checks that loans correctly trigger the player's `add_money` method and decrement bank reserves, while ignoring negative requests.
- `test_bank_summary`: Captures stdout to ensure the ledger prints properly.

Logical Bugs Found & Fixed:

- **Bug 1 (`collect` method):** The code failed to enforce its own rule ("Negative amounts are silently ignored"), blindly subtracting funds when negative numbers were passed. Fixed by prepending `if amount` $\leq$ `0: return`.
- **Bug 2 (`give_loan` method):** The code failed to deduct the loan amount from the bank's internal cash reserves (`self._funds`), creating money out of thin air. Fixed by adding `self._funds -= amount`.

3.2 Module: `dice.py` (Coverage: 100% Branches)

Test Strategy:

- `test_dice_initial_state`: Verified native instantiation at 0.
- `test_dice_roll_boundaries`: Rolled the dice 100 times to guarantee generation strictly between 1 and 6.
- `test_dice_handles_doubles` & `test_dice_resets_streak_on_non_doubles`: Forced identical and non-identical rolls to track the streak counter's incrementation and erasure.

Logical Bugs Found & Fixed:

- **Bug 1 (`roll` method):** The code was generating `random.randint(1, 5)` instead of a standard 6-sided die. Fixed the boundary to `random.randint(1, 6)`.

3.3 Module: `player.py` (Coverage: 100% Branches)

Test Strategy:

- `test_player_starts_with_correct_defaults`: Confirms natural creation yields `STARTING_BALANCE` and false jail flags.
- `test_player_adds_and_deducts_money_correctly`: Checked math and verified negative deductions throw a `ValueError`.
- `test_player_bankruptcy`: Deducted all cash to trigger the flag.
- `test_player_net_worth_includes_properties`: Created dummy properties to ensure `net_worth` sums cash plus asset values.
- `test_player_normal_move_no_go` & `test_player_movement_and_passing_go`: Verified position updates and ensured the $200 salary is exclusively awarded upon wrapping around index 0.
- `test_player_landing_exactly_on_go` & `test_player_going_to_jail`: Confirmed exact-match conditions and teleportation to index 10.
- `test_player_add_and_remove_properties`: Manipulated the property array to test length checks and list removal safety.

Logical Bugs Found & Fixed:

- **Bug 1 (`net_worth` method):** The code was simply returning `self.balance`, ignoring physical assets. Fixed by summing up the `price` of all owned properties alongside cash.
- **Bug 2 (`move` method):** The code strictly required players to land exactly on index 0 to receive the $200 salary. Fixed by triggering the payout if the new position index is mathematically lower than the old position index (indicating a wrap-around).

3.4 Module: `property.py` (Coverage: 100% Branches)

Test Strategy:

- `test_property_rent_mechanics`: Ensured standard rent processing, and that mortgaged properties explicitly charge $0 rent.
- `test_property_group_monopoly_rent_bug`: Grouped three properties and divided ownership to verify the multiplier correctly evaluates strict exclusivity.
- `test_property_group_all_owned_by_none`: Verified the Bank (None) cannot trigger monopoly multipliers.
- `test_property_mortgage_logic`: Verified math for mortgaging (50% price) and unmortgaging (cost + 10% interest).
- `test_property_is_available`: Walked a property through its full lifecycle (Unowned → Owned → Mortgaged → Repossessed) to test availability flags.

Logical Bugs Found & Fixed:

- **Bug 1 (`all_owned_by` in PropertyGroup):** The original code used `any(...)`, meaning rent doubled if a player owned just one property in a color group. Fixed by changing to `all()`.
- **Bug 2 (`is_available` in Property):** The code incorrectly checked `self.owner is None and not self.is_mortgaged`. If the bank repossessed a mortgaged property, it became permanently unbuyable. Simplified to just `return self.owner is None`.

3.5 Module: `cards.py` (Coverage: 100% Branches)

Test Strategy:

- `test_deck_draw_and_peek`: Created a targeted dummy deck. Tested `peek()` to ensure it doesn't advance the index, and `draw()` to ensure sequential pulling and loop-back behaviors.
- `test_deck_reshuffle`: Advanced halfway through the deck and called `reshuffle()` to ensure the index reset to 0.
- `test_deck_empty_deck_crash_bugs`: Intentionally initialized a deck with zero cards to force boundary failures.

Logical Bugs Found & Fixed:

- **Bug 1 (ZeroDivisionError crashes):** The code used a modulo operator on `len(self.cards)`. Empty decks caused a literal math crash (`modulo by zero`). Fixed by adding a `if not self.cards: return 0` guard to handle empty lists gracefully.

3.6 Module: `board.py` (Coverage: 100% Branches)

Test Strategy:

- `test_board_get_property_at` & `test_board_get_tile_type`: Checked if specific board indexes return the correct mapped tile names (e.g., verifying index 12 returns "blank" for missing utilities).
- `test_board_is_purchasable_logic`: Confirmed Go (0) cannot be bought, while standard real estate can.
- `test_board_ownership_queries`: Simulated heavy property purchasing and checked if owned vs. unowned global counts inversely updated correctly.

Logical Bugs Found & Fixed:

- **Bug 1 (`is_purchasable` constraint failure):** Had a redundant secondary check banning the purchase of mortgaged properties from the bank. Deleted the constraint, simplifying it to `return prop.owner is None`.

3.7 Module: `game.py` (Coverage: 97% Branches)

Test Strategy:

- Opted for strict native interactions without excessive `MagicMock` to maintain structural authenticity during testing.
- Simulated precise `monkeypatch` input queues arrays to navigate heavily nested `while` loops for Auctions, Trading, Mortgages, and Jail mechanics.
- Achieved a natively high 97% coverage on a massive 500-line monolithic engine, dropping only isolated `return` sanity checks on unhittable string-casting failures.

Logical Bugs Found & Fixed:

- **Bug 1 (`buy_property` affordability trap):** The logic used $\leq$ instead of $<$, banning purchases that perfectly zeroed out a player's balance. Corrected to allow exact-fund purchases.
- **Bug 2 (`find_winner` reversed):** The engine searched for the winner using `min(self.players...)` instead of `max`. The system literally crowned the poorest player. Fixed to `max()`.

- **Bug 3 (`trade` destroys money):** The trading engine correctly deducted cash from the buyer, but never added it to the seller, causing the money to vanish from the economy. Fixed by adding `seller.add_money()`.
- **Bug 4 (`pay_rent` destroys money):** The exact same void logic plagued regular rent payments. Fixed by chaining `prop.owner.add_money(rent)` to the transaction.